

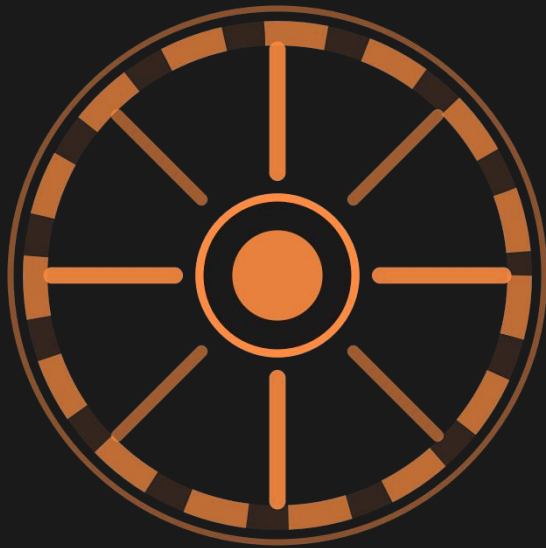
第二节

AI 时代程序员的核心竞争力壁垒构建和 Oryx 社区

进入具体技术之前，先回答程序员心里最痛的问题：
AI 时代我的核心竞争力是什么，我该怎么办。
这节课给三层递进答案，以及一个长期成长入口。

《企业级AI编程实战营》· 竞争力 + 壁垒 + 社区

> @Robert/Oryx Labs



"AI 这么强 我作为程序员，还有什么价值？"

表面问的是职业前景

实际问的是认知坐标

你过去衡量自己价值的标尺，在 AI 时代还管用吗？

这节课给你一个新标尺

本节课做四件事

01

看清真实状态

数据 · 大佬在做什么 · 认知区别

不是听宣传，是看真实数据和有实战产出的人具体在做什么。

02

三层竞争力

用熟 AI + 工程纪律 + 复利型资产

三层递进，不是并列。每一层都有具体的判断标准和落地路径。

03

怎么卷，卷对方向

四个真实建议 · 业务 vs AI 开发

开源是最好的载体，AI 时代红利在放大，落到日常五件事。

04

Oryx 社区

四个项目 · 长期共建入口

一个能持续练下去的环境，真正的竞争对手是你身边那个人。

什么不再是立身之本

它们没消失，没作废。但都从"决定你值多少钱"变成了"决定你的下限"。

入场券 工具熟练度

"我会用 Claude Code / Cursor / Copilot"这件事在 2026 年已经不构成壁垒。AI 工具迭代速度远快于人的学习速度。工具熟练度是入场券，所有人都会有一张。业界称之为：智力平权。

入场券 技术栈深度

"我精通 Java / 深耕 Rust"的溢价在缩水。AI 能帮你在几天内跨进一个新技术栈。真正值钱的不是深度本身，是你在这门技术上踩过坑形成的工程直觉——那些 AI 帮你跨进来但替代不了的东西。

入场券 工作年限

"我做了十年开发"的护城河在变小。一个用 AI 用熟的三年工程师，某些场景能输出过去八年工程师才有的产出。重要的不是你做了多少年，是你在这些年里沉淀了什么。

入场券 大厂背景

"我在大厂干过"的真实价值从来不是公司名字，是"我在这里见过什么、做过什么、积累了什么判断"。AI 时代，这些判断的传递速度变快，不在大厂的工程师也可以快速获得类似视野。

数据不骗人：三组有信誉的数据

84% / 29%

使用率 / 信任度

Stack Overflow 4.9万人调研 2025

84% 的开发者在用 AI，但信任度从 40% 跌到 29%。46% 主动不信任 AI 准确性。66% 说花了更多时间在修"差不多对但不全对"的代码。这叫 adoption-trust gap。

-19%

实际比预期慢

METR 随机对照实验 2025

开发者以为 AI 让自己快了 20%，实际上慢了 19%。差距 39 个百分点。专家的预测也错了。时间花在写 prompt、等输出、修错误、context switching 上，超过了 AI 节省的时间。

-20%

22-25岁开发者岗位

Stanford 数字经济研究 2025

22-25 岁软件开发者就业相比 2022 年峰值下降近 20%。欧洲初级技术岗位下降 35%。初级岗位被实质性挤压是真实的结构性变化，不是炒作。

使用率上升的同时，信任度在下降。行业处于"全员上路、半信半疑、效果不明"的混乱过渡状态

大佬们具体在做什么（不是说什么）

Karpathy

OpenAI 联合创始人

承认"我从未感到这么落后", 同时坚持亲手做 nanochat, 守住"建不了就不理解"原则, 每天跟踪 16 个新工程概念。

DHH

Rails 创始人

半年内 180° 转向——从"AI让能力从手指流出去"到"现在是最激动人心的事"。转向原因: 换了工具姿势, 从 IDE 内嵌换成 terminal agentic。

Simon Willison

Django 联合创始人

每天用 AI 写代码并记录实践。总结出 vibe engineering: 加自动化测试+规划+文档+code review, 然后用 AI 加速这一切。

Boris Cherny

Claude Code 创造者

团队上月 200 个 PR 全部 AI 写, 但同时说: "最终是人在做这件事。AI 是坐在你旁边的专家, 不是替代你的机器"。

五个共同点: 亲手用 · 持续调整 · 不卖答案 · 守住工程理解 · 警惕宣传

核心区别：vibe coding vs agentic engineering

vibe coding

- 描述想要 X，接受 AI 的输出
不深究 AI 怎么实现的、有没有问题
- 适合：个人项目、原型验证、低风险内部工具
不适合：生产系统、企业产品、有用户依赖的服务
- 风险：vibe coded 的 PR 经常缺失 edge case、错误路径、异常逻辑，因为开发者没有规定它们
重度用 AI 编程的团队，PR review 时间上升了 91%
- 你的天花板 = 你对系统理解的天花板
AI 不会替你理解系统

agentic engineering

- 把工程判断编码进 AI 的工作流
让 AI 在严格约束下高效产出
- CLAUDE.md + Skills + Hooks + Plan Mode
护栏设好了，AI 才能真正跑起来
- 系统理解不让渡
知道系统怎么坏、edge case 在哪、failure mode 是什么
- 适合任何场景
门槛：工程师本身要有判断力和经验

DHH 那句话没错：如果你只能 vibe code，你就不是程序员——这不是说代码，是说工程。

AI 时代程序员三层竞争力

三层递进，不是并列。没有下层，上层是空谈。三层同时在，才是真正立得住的护城河。

1 第一层

用熟 AI

workflows 设计能力

设计人和 AI 协作的完整工作流。工具越强，设计工作流的价值越大。同一把工具，用法差异决定产出差几个数量级。

2 第二层

工程纪律

深度工程判断力

在关键节点做对的技术判断。AI 让"做事"变快，"做对的事"反而成了瓶颈。判断力的价值在 AI 时代被进一步放大。

3 第三层

复利型资产

复利资产积累能力

每个项目结束后，你沉淀了什么。做了项目不等于沉淀了资产。这一层最被忽视，但回报最大。

判断立身之本的三条标准

三条都满足，才是真正能站住的东西。缺任何一条，都有短板。

01 AI 难以复制

AI 做不到你这个深度、带不出你这个判断

判断、决策、取舍、拍板，这一层在可见的未来仍然是 AI 难以替代的。不是因为 AI 笨，是因为这一层的输入是真实业务语境、具体团队约束、隐性历史包袱，AI 拿不到完整。

02 你独有

你做事的方式有你的指纹

不是"行业普遍能力"，是你的具体经历、具体踩坑、具体判断风格决定的能力。别人没走过你走过的路，这些复制不了。

03 能持续生长

会随着你做事不断累积，不是一次性资产

你的 Skills 库、开源贡献、踩坑笔记，用得越多越值钱，而不是越用越少。这是复利的底层逻辑。

只满足两条不够：AI 难复制+你独有但不生长=过去式资产；三条都满足才是真正的护城河

第一层： workflow 设计能力

不是“你会用 AI 写代码”，是你能设计一套人和 AI 协作的工作流，让一件事从模糊需求到生产上线全程跑通

```
bash - 80x24

$ /specify
└─ AI 追问边界条件，输出可执行规格

$ /plan
└─ 生成技术方案，等你 review 确认

$ /tasks
└─ 拆解为独立可验收任务列表

$ /implement
└─ 逐任务执行，每步可见可控
```

为什么是立身之本

三条标准全过。AI 不能给自己设计工作流；每人的设计带个人指纹；做的项目越多，工作流越精炼。

为什么工具越强价值越大

工具弱时大家都只能手撸，差距不大。工具强了，用法差异成了产出差距的最大变量。同一需求，工作流好的两小时做完，不好的两天还在拉扯。

跨场景迁移性极强

Java 项目的工作流稍作调整就能用在 Rust 项目。底层模式是稳定的：理解 → 约束 → 验证 → 沉淀，不依附在任何技术栈上。

这也是业界涌现这么多 SDD 工具和 Skills 仓库的原因——大家都想把工作流固化下来。

第二层：深度工程判断力

不是"你会写代码"，是你能在关键节点做出对的技术判断——这个项目选 A 还是 B？这段代码该不该上生产？
这个 bug 是表象还是底层问题？

为什么 AI 难以复制

判断依赖具体语境和具体踩坑。AI 能给你方案对比，不能替你拍板。拍板需要的输入是你这家公司的真实状况、你这个团队真实能力、你这个时点的真实优先级——AI 拿不到完整。

判断力带个人风格

偏保守的工程师遇到不确定先停下来求证；偏激进的先做出来再说；偏长期的任何决定都问五年后怎么看。这些风格是个人指纹，AI 复制不了。

AI 时代判断力的价值在被放大

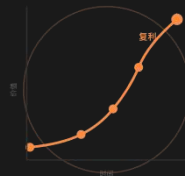
AI 让"做事"变快了，"做对的事"反而成了瓶颈。以更快的速度做出了更多错的东西，沉没成本反而更大。速度被 AI 提上去了，判断必须跟着提上去，否则会出大事。

判断力能持续生长

每一个项目都在累积你的判断库。这次选错了，下次就不会再选；这次选对了，下次遇到类似场景就有锚点。资深工程师的真正价值在"做过多少判断，命中率有多高"。

第三层：复利资产积累能力

做了项目不等于沉淀了资产。没有主动沉淀的项目，做完就结束了，留下的只是工资条上几个数字。



本地资产

CLAUDE.md · Skills
库 · Slash
Commands · Hooks 配
置 · 项目脚手架

半公开资产

团队内部技术文章 · 踩坑分
享 · 决策记录

公开资产

技术博客 · 开源贡献 · PR
记录

身份资产

GitHub 主页 · 技术 IP · 社
区影响力

AI 大幅加速沉淀

过去沉淀一份高质量资产要一晚上，现在可能只要一小时。AI 不会替你产生资产，但 AI 能让你产生资产的速度变快好几倍。能用上这个加速的人，资产积累速度会甩开不用的人很大一截。

最有价值的是公开+身份资产的合体：开源贡献。下一页专门讲。

复利资产的最佳载体：开源

开源是程序员个人价值在 AI 时代最被低估的杠杆

L1

能力收益

最快见效，1-2个月可感受

更高标准的代码库 · 免费高质量 code review · 直接看顶级工程师怎么思考。

L2

身份收益

GitHub 是程序员的公开简历

你的 commits 是公开的、可验证的 · 招聘方一眼能看到你做过什么 · 代码替你说话，不需要解释自己。

L3

网络收益

进入高质量工程师圈子

maintainer 会记住认真做事的人 · 圈子关系跨越公司、行业、时间 · 最好的机会很多不是投简历来的。

L4

复利收益

最长期、最值钱

五年前贡献的代码，五年后还在被人用 · GitHub 主页随时间持续累积 · 大多数工作的价值随时间衰减，开源贡献的价值随时间增值。

能力层最快见效，大家都看得见；真正值钱的是后三层——身份、网络、复利。公司给不了你这三样。

AI 时代，开源红利在放大

AI 编程没有让开源贬值，反而让开源对程序员个人价值的红利在放大。四件事同时发生，过去几十年从未有过。

01 门槛大幅降低

AI 帮你读不熟悉的代码库、跨入不会的语言、起草 PR 描述、回应 review comments。过去拦住一大批"想做但不知怎么开始"的工程师的具体障碍，被大幅降低了。

02 含金量在提升

"会某某框架"已经不能证明什么，但 GitHub 上的开源贡献不一样——它是被严格 review 通过的真东西。maintainer 认证了你的代码够格，这种"被认证的真"在 AI 时代变得稀缺。

03 复利速度在加快

过去一个 PR 可能需要几周；现在读懂代码更快、写实现更快、测试更全、PR 描述更清晰。同样时间投入，产出几倍于以前。晚开始的人，有可能更快追上。

04 精英被更快识别

AI 时代圈子里"能做事的人"反而更稀缺。AI 让噪音增多，但 maintainer 能很快判断哪些 PR 是真做事、哪些是包装。真做事的工程师，比过去更容易被识别和记住。

怎么卷，卷对方向：四个真实建议

01 每个项目跑完整 workflows，不偷懒

不要从"开始写代码"开始，从"理清需求 + 写 spec"开始；不要"代码写完就结束"，继续跑测试、部署、上线、监控。同一套 workflows 跑过三个不同性质的项目，才知道哪些环节是通用的。

02 每个关键决策都问"为什么"，记录下来

选 A 不选 B 的真实工程权衡是什么？这个判断五年后回头看会觉得对吗？没有真实踩坑的判断只是知识不是判断。复盘要落到"我下次怎么做不一样"。

03 每个项目结束主动问"沉淀了什么"

这次的代码有没有可抽出来的模板？这次解决的问题有没有可写成 Skill 的方法？这次的踩坑有没有可贡献给开源的部分？判断复利资产的标准只有一条：下次起点更高。

04 主动给自己挑一个开源项目

不一定最热门，挑你日常用到的、真心有兴趣的。从修文档、修小 bug、补测试用例开始。最大的门槛永远是第一步，后面会越来越顺。

这四件事不是额外的训练，是在每天的工作里同时建立三层立身之本

业务开发 vs AI 开发：全都要，不是选择题

业务开发

深度工程判断力的根基

- 你的判断力来自真实业务场景的摸爬滚打
- vibe code 的天花板是你的业务理解的天花板
- AI 不在场，不知道你公司的真实约束和历史包袱
- 没有深度业务经验，很难写出真正有价值的 Spec
- 这是 AI 放大不了的部分，只能靠你自己



AI 驾驭能力

把你的业务判断力放大的杠杆

- workflow 设计能力让全链路跑通
- Spec-Kit 把业务需求变成 AI 能消化的规格
- Harness 保证 AI 在你定义的边界里工作
- 复利资产把你的判断沉淀成可复用工具
- 让你用更短的时间做出更复杂的系统

业务理解是判断力的根基，AI 驾驭是放大判断力的杠杆。缺了哪个，另一个都发挥不出真正的价值

落到日常：五件事

01 每天找半小时，在不重要的项目里真正用 AI

不重要是关键词。副项目、想做但没做的 idea。光看文章没用，所有内容都不如亲手用一次。连续比偶尔有效一百倍。

02 找一个比你早一步的人，问他怎么做

不要找最远的大佬，找你身边比你早走半步的同事或朋友。他们最近遇到的坑、踩过的雷，跟你最相关。

03 不要等"看清楚再动手"

工具每三个月翻一遍，模型每六个月跳一档。业界大佬都没看清楚，他们的做法是先动手边动手边校准。这是这个时期唯一可行的姿势。

04 守住你对工程的理解，不让渡给 AI

工程的核心是理解系统在做什么、会怎么坏、用户是谁。如果你发现自己开始失去这个理解，说明你用 AI 用错方向了。

05 接受焦虑期，它是正常的

Karpathy 说"从未感到这么落后"，OpenAI 联合创始人都在焦虑，你焦虑很正常。但他在焦虑的同时在做 nanochat——焦虑唯一的解药是动手。

替代我们的不是 AI，是更会使用 AI 的人。你不需要跑赢 AI，你只需要比你的同事早一步把 AI 用起来

为什么做 Oryx 社区

光做课不够。课程结束了，人就散了。但程序员真正长进的方式不是听几节课，是长期练，跟人聊。

熟能生巧需要环境

这个行业最有趣的从来不是工具本身，而是用工具的人。一个人在家里学 AI 编程，半年下来很容易放弃。有一群人陪着练，是大多数程序员能坚持下来的唯一方式。

真正的竞争对手就在你身边

你工位旁边那个用了半年 Claude Code、攒了十几个 Skill、每周花一两小时学新工具但从来不说的人——他在前进。Oryx 是一个能持续练下去、找到同路人的地方。

我们自己也需要这个地方

训练营不是“我们已经做成功了来分享”，是“我们在一线还在折腾，把这段路上的发现和你一起走一遍”。认知建立之后，练练练才是真正加深认知的地方。



Oryx 社区：四个真实项目

同一套工作流，在四种不同场景里跑四遍。跑一次只是体验，跑四次才是肌肉记忆。

01

When

分布式延时组件

新项目 · 分布式系统

延时投递是企业到处都有的真实需求（订单超时、优惠券过期、定时通知）。涉及 gRPC · etcd · 集群构建 · 时间轮。既练 AI 编程，又得到真实分布式系统项目经验。

02

DifyPro

Dify 二次开发

改造老项目 · 大型代码库

企业有大量 Dify 内部二开的真实需求，业界缺标准答案。练 AI 编程在 100 万行陌生代码里的协作。学了就可以直接用到自己公司。

03

OryxOS

AI 智能体操作系统

系统级项目 · 从 0 到 1

成熟的 Agent OS 是接下来每个企业的标配。深度对比 Hermes Agent、OpenClaw 架构设计。学员既能自己写，也能基于开源做二次开发。

04

mq9

Rust 写的 Agent 通信基础设施

真实开源贡献 · 陌生语言

真实活跃的开源项目，自身目标是进 Apache 顶级项目。体验给有真实影响力的开源项目贡献的全流程：选 issue · 提 PR · 被 maintainer review。

四个项目放在一起，是在不同场景里反复训练同一套工作流——让它变成肌肉记忆。

本节总结 + 下节预告

这节课建立了

▸ 真实状态

数据不骗人 (84%/29%, -19%, -20%) · 大佬在做什么 · vibe coding vs agentic engineering。

▸ 三层竞争力

workflow 设计 + 工程判断力 + 复利资产。开源是最好的载体，AI 时代红利在放大。

▸ 卷对方向

四个真实建议 + 业务和 AI 开发全都要 + 五件日常事项。

▸ Oryx 社区

四个真实项目跑四遍，熟能生巧需要环境。

第三节预告

▸ Agent OS 是什么

2026 年企业级 AI 落地的标配，也是训练营最重的项目。

▸ 五组件拆解

LLM + Tools + Memory + Loop + Environment——Agent 在技术上到底是什么。

▸ 主流 Agent OS 对比

Hermes Agent vs OpenClaw 的设计哲学差异。

▸ OryxOS 完整设计方案

架构图 · Agent 进程模型 · 运行时核心 · 沙盒隔离。

看清楚这个转向的人会拿到这波最大的红利，看不清楚的会在旧坐标里挣扎被悄悄甩在后面